

Desarrollo de Software. Escenario 1

Desarrollo del software de una planta automatizada de paletizado

Proyectos y organización de escenarios II

ÍNDICE

- 1. Descripción de la tarea a realizar**
- 2. Explicación de la planta automatizada**
- 3. Escenario 1; Conexión web con la base de datos mediante MQTT y ESP32**
 - 3.1 Creación de la página web y la base de datos
 - 3.2 Requisitos previos para el correcto funcionamiento de la tarea
 - 3.3 Implementación final del escenario; conexiones.

Creado por:

Ainhoa García Herrera

Valentina Monteserin Luna

Rihab Baitar

Claudia Castillo Suarez

DESCRIPCION DE LA TAREA A REALIZAR

En la asignatura de Proyectos 2 (PR2), se propuso la realización de un proyecto grupal orientado al diseño e implementación de escenarios de integración aplicados a la automatización industrial. El objetivo principal de esta actividad es poner en práctica los conocimientos adquiridos en distintas asignaturas del curso, especialmente en Programación Avanzada (PRA) y Gestión de Datos para la Industria (GDI), mediante el desarrollo de una solución propia y original.

Para ello, cada grupo ha debido de diseñar e implementar dos escenarios de integración funcionales en los que diferentes dispositivos, robots y aplicaciones informáticas intercambien información y se coordinen para ejecutar tareas automatizadas.

Para comprender totalmente las soluciones propuestas por el grupo, es imprescindible conocer primero en qué consiste nuestra propuesta de automatización

EXPLICACIÓN DE LA PLANTA AUTOMATIZADA

Nuestra propuesta se basa en el diseño de una planta automatizada de empaquetado de bombones, la cual trabaja con tres sabores diferentes y contempla dos modalidades de compra.

La primera modalidad está orientada a pedidos industriales o al por mayor, donde el cliente debe adquirir un mínimo de dos cajas y un máximo de cuatro cajas de un mismo sabor. Esta opción está pensada para pedidos de gran volumen y producción más estandarizada.

Por otro lado, la segunda modalidad corresponde a los pedidos personalizados, destinados a ventas de menor escala. En este caso, una misma caja de gran tamaño puede incluir distintas combinaciones de sabores, permitiendo al cliente seleccionar la distribución de bombones según sus preferencias.

En cuanto a la planta automatizada, esta dispone de diferentes sistemas de control de calidad distribuidos a lo largo de todo el proceso de producción, con el objetivo de verificar el correcto funcionamiento de la línea y garantizar la calidad final del producto.

Además, la instalación incorpora diversas medidas de seguridad industrial orientadas a proteger tanto a los operarios como a los equipos de trabajo. Entre ellas, destacan las celdas de seguridad destinadas al aislamiento de robots que manipulan cargas elevadas, así como sistemas de control y limitación de velocidad en los cobots, capaces de detectar la presencia

humana y adaptar su funcionamiento para reducir riesgos durante la interacción con los operarios.

Una vez descrito el funcionamiento general de la planta automatizada, se presentan los dos escenarios de integración desarrollados como parte de la solución propuesta.

El primer escenario se centra en la creación de una página web dinámica conectada a una base de datos, permitiendo la gestión y actualización de información en tiempo real mediante scripts desarrollados en Python y el uso de tecnologías de comunicación basadas en MQTT. Este escenario tiene como objetivo integrar la interacción del usuario con el sistema automatizado, facilitando la gestión de pedidos y la comunicación entre los distintos componentes de la planta.

Por otro lado, el segundo escenario se enfoca en la implementación de técnicas de programación avanzada junto con el uso de MQTT y MQTTX para gestionar de manera eficiente la recepción y procesamiento de señales dentro de RoboDK. Gracias a esta integración, es posible coordinar correctamente el inicio y ejecución de los procesos automatizados de la planta, asegurando una sincronización adecuada entre los diferentes dispositivos y sistemas involucrados.

En este primer documento, se estudiará el desarrollo del primer escenario.

ESCENARIO 1; Conexión web con la base de datos mediante MQTT y ESP32

En primer lugar, para el desarrollo de este escenario ha sido necesario diseñar e implementar una página web que actúe como portal principal de nuestro negocio, permitiendo a los clientes realizar pedidos de manera sencilla e intuitiva. Esta plataforma sirve como punto de interacción entre el usuario y el sistema automatizado de la planta, facilitando la gestión de compras y el envío de información hacia el resto de los componentes integrados en la solución.

Por otro lado, se ha diseñado e implementado una base de datos relacional en PostgreSQL, en la cual se almacenan los datos recibidos desde la página web. Para ello, se han creado diferentes tablas destinadas a la gestión de clientes, pedidos, envíos, proveedores y otros elementos relacionados con el funcionamiento de la planta automatizada.

Cabe destacar que esta base de datos será utilizada también en el desarrollo del proyecto final, por lo que actualmente existen apartados y estructuras que todavía no han sido completados en

su totalidad, ya que dependen de funcionalidades y escenarios que se implementarán en fases posteriores del proyecto.

La conexión entre la página web y la base de datos no se realiza de forma directa, sino mediante una capa intermedia desarrollada en Python denominada *mqtt_bridge.py*, cuya función principal es actuar como intermediario y coordinador del sistema. La página web envía la información de los pedidos mediante MQTT al tópico *tienda/pedido/nuevo*, desde donde el script recibe los datos en formato JSON para iniciar su procesamiento.

Una vez recibido el mensaje, el programa valida la información correspondiente al cliente y a los productos solicitados, determina automáticamente el tipo de pedido y comprueba la disponibilidad de stock. Si todas las verificaciones son correctas, el sistema procede al almacenamiento de la información en la base de datos PostgreSQL.

El script desarrollado ejecuta varias operaciones encadenadas de manera automática. En primer lugar, crea o actualiza los datos del cliente en la tabla correspondiente; posteriormente registra la cabecera del pedido, inserta las líneas de productos asociadas, genera la información inicial del envío y actualiza el stock disponible de los diferentes sabores. Todas estas operaciones se realizan dentro de una transacción de base de datos, garantizando así la integridad de la información. De este modo, si se produce cualquier error, el sistema revierte automáticamente todos los cambios realizados, evitando inconsistencias en la base de datos.

Además, el sistema incorpora un mecanismo de comunicación bidireccional mediante MQTT. En caso de error, se envían mensajes de alerta a los sistemas correspondientes, mientras que, si el proceso finaliza correctamente, se generan mensajes de confirmación dirigidos a los distintos módulos relacionados con la tienda, la logística y la fábrica.

```
PS C:\Users\ainho\Desktop\Proyecto_PR2_final> python mqtt_bridge.py
DB conectada
Conectando MQTT a kf12786f.ala.us-east-1.emqxsl.com:8084 ...
MQTT OK. Suscrito a tienda/pedido/nuevo
```

Uno de los aspectos más relevantes relacionados con la asignatura de Programación Avanzada (PRA) ha sido la implementación de una estructura lógica basada en combinaciones de pedidos. Dado que la planta trabaja con tres sabores distintos y que los pedidos industriales permiten entre una y cuatro cajas, se generan un total de 34 combinaciones posibles con repetición. Para gestionar esta situación de forma eficiente, se diseñó una estructura organizada de combinaciones, similar a un árbol o sistema de mapeo, capaz de transformar automáticamente cada pedido en un identificador único denominado *numero_combi*.

Este identificador, comprendido entre los valores 1 y 34, se almacena en la base de datos y, al mismo tiempo, se publica mediante MQTT a través de topics. De esta forma, dispositivos como la ESP32 pueden recibir y mostrar dicha información, mientras que RoboDK utiliza el valor de *numero_combi* como disparador automático de procesos, seleccionando la rutina de *pick and place* correspondiente según el tipo de pedido y la combinación solicitada. Esto permite

establecer una integración completa entre la plataforma web, la base de datos y el sistema robótico de la planta automatizada.

En conclusión, la integración entre la web y la base de datos se ha desarrollado siguiendo una arquitectura orientada a eventos basada en MQTT y Python, complementada con control transaccional sobre PostgreSQL y una capa de lógica avanzada encargada de gestionar las combinaciones de pedidos. Este enfoque proporciona una solución modular, escalable y fácilmente adaptable a futuras modificaciones tanto en los procesos de automatización como en las configuraciones de producción de la planta.

REQUISITOS PREVIOS PARA LA IMPLEMENTACION DE LA TAREA

Para garantizar el correcto funcionamiento e integración de todos los componentes del sistema, es necesario disponer previamente de diferentes herramientas y tecnologías instaladas y configuradas adecuadamente. Entre ellas, destacan PostgreSQL y pgAdmin para la gestión y administración de la base de datos relacional, Python para el desarrollo de los scripts encargados de la lógica de integración y comunicación, MQTT como protocolo de mensajería utilizado para el intercambio de información entre dispositivos y servicios, y Arduino IDE para la programación y configuración de los dispositivos ESP32 empleados en el proyecto.

El proceso de una correcta instalación es el siguiente:

Una vez abierta la base de datos (formato backup custom) en PGAdmin, y conectado al broker de MQTT con los siguientes credenciales:

Name:	Broker EMQX Cloud
Wss://	kf12786f.ala.us-east-1.emqxsl.com
Port:	8084
Path:	/mqtt
Username:	web_choco
Contraseña:	asdfghjPOIU2345678

Y suscrito a tarea\#, logistica\#, numero_combi\# y frabrica\# se debe hacer lo siguiente:

Primero, abrir el CMD dentro de la carpeta donde hemos descomprimido todos los archivos

PC (puente)

1. Copia .env.example a .env (copy .env.example .env) y rellena PGPASSWORD, PGDATABASE. En el env.example ya se encuentra completado, siendo lo unico

necesario a poner la contraseña de postgresSQL del usuario para poderlo conectar ala base de datos.

Importante, el nombre de nuestra base de datos se llama PLanta_PR2. En caso de cambiarle el nombre tambien es necesario hacerlo en el .env

2. Instala dependencias:

```
python -m pip install -r requirements.txt
```

3. Ejecuta el puente (dejar abierto):

```
python mqtt_bridge.py
```

4. No se debe ejecutar otro puente a la vez sobre el mismo topic.

ESP32

1. Abre esp32/choco_esp32_mqtt/choco_esp32_mqtt.ino.
2. Cambia WIFI_SSID y WIFI_PASSWORD.
3. **Broker:** mismo host/usuario/clave que la web. Este sketch usa **MQTT TLS puerto 8883** (mqtts). Se debe activar ese listener en EMQX Cloud si no está (la web sigue usando **WSS 8084**).
4. Por defecto está suscrito a `fabrica/#`. El Python publica en `fabrica/esp32/pedido_guardado` tras guardar un pedido
5. **Monitor serie** 115200 baud:

PAGINA WEB

Como la web es local, es necesario descargar la carpeta entera y acceder mediante ella.

Al ser una simulación, en el envío, no es necesario que las credenciales sean reales

Como conclusión, el objetivo principal de este escenario ha sido diseñar e implementar una arquitectura completa que permita la conexión entre la web y la base de datos mediante MQTT, garantizando un sistema robusto, estructurado y trazable.

Más allá del simple intercambio de información, se ha planteado un flujo de trabajo automatizado en el que cada pedido realizado desde la web sigue una secuencia controlada: recepción del mensaje, validación de los datos, comprobación del stock disponible, almacenamiento en la base de datos y notificación a los módulos correspondientes de producción o logística.

Gracias a esta integración, se consigue uno de los objetivos clave del proyecto: disponer de un control de stock en tiempo real, permitiendo determinar con precisión si un pedido puede ser procesado o si debe ser rechazado o detenido por falta de recursos.

Asimismo, el sistema queda preparado para su futura integración con RoboDK. La generación de indicadores estructurados (como el identificador del pedido o variables de control) permite transformar una orden comercial en una señal técnica interpretable por los sistemas robotizados, facilitando la ejecución, pausa o redirección de procesos de pick & place en función del estado real de la producción. De este modo, no solo se automatiza el inicio de las tareas, sino también su supervisión y control ante posibles incidencias.

En definitiva, este escenario establece una base sólida para un entorno industrial conectado, donde la información fluye de forma coherente entre sistemas, la comunicación es desacoplada y el sistema resultante es escalable, permitiendo evolucionar hacia una orquestación completa de procesos automatizados con RoboDK, tanto en la ejecución como en la gestión segura de la producción.